

Instituto Tecnológico Superior de Lerdo



Nombre del Alumno
Miguel Angel Luna Camacho

Numero de Control

252310055

Docente: Ing. Jesús Salas Marín

Actividad Integradora - Arreglos unidimensionales

Lerdo, Dgo., a 22 de Mayo de 2026

Actividad

Desarrollar un script en PHP que cumpla con los siguientes requisitos:

1. **Capturar datos** de al menos 5 productos (nombre y precio).
 - Debes capturarlos desde un formulario html (usa index.php para ello).
 - Debes utilizar validación básica desde html.
2. **Almacenar los datos** en dos arreglos unidimensionales paralelos en un script aparte (ej., usando un archivo procesar.php como se vio en clase):
 - \$productos[].
 - \$precios[].
3. **Calcular y mostrar:**
 - El **precio total** de todos los productos investiga como usar array_sum().
 - El promedio de precios. o El producto más caro y su precio investiga como usar max(). o
 - El producto más barato y su precio investiga como usar min().
4. Mostrar los resultados con un formato claro y ordenado en una tabla HTML.

index.php

```
147 <body>
148
149 <header>
150   <h1> TiendaPHP</h1>
151   <p>Módulo de Gestión de Inventario · Actividad Integradora P00</p>
152 </header>
153
154 <!--
155   El formulario envía los datos mediante POST hacia procesar.php.
156   La validación básica se realiza con los atributos HTML5:
157       required → campo obligatorio
158       min      → precio mínimo permitido ($0.01)
159       step     → acepta decimales
160 -->
161 <div class="card">
162   <h2>Registro de productos</h2>
163
164   <form action="procesar.php" method="POST">
165
166     <?php
167     /*
168      * Generamos dinámicamente 5 filas de productos
169      * usando un bucle for, así el código no se repite.
170      */
171     for ($i = 1; $i <= 5; $i++):
172     ?>
173
174     <div class="row-number">Producto <?= $i ?></div>
175
176     <div class="product-row">
177
178       <!-- Campo: nombre del producto -->
179       <div class="field-group">
180         <label for="nombre_<?= $i ?>">Nombre</label>
181         <input
182           type="text"
183           id="nombre_<?= $i ?>"
184           name="nombre[]"
185           placeholder="Escribe un producto"
186           required
187           maxlength="60"
188         />
189       </div>
```

```

190
191     <!-- Campo: precio del producto -->
192     <div class="field-group">
193         <label for="precio_<?= $i ?>">Precio ($)</label>
194         <input
195             type="number"
196             id="precio_<?= $i ?>"
197             name="precio[]"
198             placeholder="0.00"
199             required
200             min="0.01"
201             step="0.01"
202         />
203     </div>
204
205 </div><!-- /.product-row -->
206
207 <?php if ($i < 5): ?>
208 |   <hr class="divider" />
209 <?php endif; ?>
210
211 <?php endfor; ?>
212
213 <button type="submit" class="btn-submit">
214 |   ▶ Procesar inventario
215 </button>
216
217 </form>
218 </div>
219
220 <footer>
221 |   Programación Orientada a Objetos · Ingeniería Informática
222 </footer>
223
224 </body>
225 </html>

```

fin index.php

Formulario con 5 filas generadas por un bucle for. Usa validación HTML5 (required, min="0.01", step="0.01") y envía los datos por POST.

procesar.php

```
codigo > 🐛 procesar.php
1  <?php
2  /*
3   * Descripción: Recibe los datos enviados desde index.php (método POST),
4   *               los almacena en dos arreglos paralelos ($productos y $precios),
5   *               realiza los cálculos estadísticos requeridos y
6   *               guarda todo en variables de sesión para pasarlos a resultados.php.
7   */
8
9  // — 1. Iniciar sesión para poder compartir datos entre páginas —————
10 session_start();
11
12 // — 2. Validación de servidor: verificar que los datos llegan por POST —————
13 if ($ _SERVER['REQUEST_METHOD'] !== 'POST'
14     || empty($_POST['nombre'])
15     || empty($_POST['precio'])) {
16
17     // Si alguien intenta acceder directamente, lo redirigimos al formulario
18     header('Location: index.php');
19     exit;
20 }
21
22 // — 3. Recuperar los arreglos enviados por el formulario —————
23 /*
24  * Como el formulario usa name="nombre[]" y name="precio[]",
25  * PHP los recibe automáticamente como arreglos indexados.
26  */
27 $nombres_raw = $_POST['nombre']; // arreglo con los nombres (strings)
28 $precios_raw = $_POST['precio']; // arreglo con los precios (strings numéricas)
29
30 // — 4. Inicializar los arreglos paralelos de trabajo —————
31 $productos = []; // almacenará los nombres saneados
32 $precios = []; // almacenará los precios como floats válidos
33
34 // — 5. Recorrer, sanear y poblar los arreglos —————
35 /*
36  * Iteramos con array_keys() para garantizar que usamos los índices
37  * originales; así ambos arreglos siempre están sincronizados (paralelos).
38  */
39 foreach (array_keys($nombres_raw) as $i) {
40
41     // Sanear el nombre: eliminar espacios extra y caracteres peligrosos
42     $nombre = trim(htmlspecialchars($nombres_raw[$i], ENT_QUOTES, 'UTF-8'));
43
44     // Convertir el precio a float y verificar que sea un número positivo
45     $precio = floatval($precios_raw[$i]);
46
47     // Solo agregamos la fila si ambos campos son válidos
48     if ($nombre !== '' && $precio > 0) {
49         $productos[] = $nombre; // agregar al arreglo de nombres
50         $precios[] = $precio; // agregar al arreglo de precios
51     }
```

```

53
54 // — 6. Segunda validación: asegurarnos de tener al menos 1 producto válido —
55 if (empty($productos)) {
56     header('Location: index.php?error=datos_invalidos');
57     exit;
58 }
59
60 // — 7. Cálculos estadísticos —————
61
62 /*
63  * array_sum($precios)
64  * Suma todos los elementos del arreglo en una sola instrucción.
65  * Es más eficiente y legible que un bucle for manual.
66  */
67 $total = array_sum($precios);
68
69 /*
70  * Promedio: total dividido entre la cantidad de productos.
71  * count() devuelve el número de elementos del arreglo.
72  */
73 $cantidad = count($precios);
74 $promedio = $total / $cantidad;
75
76 /*
77  * max($precios) → devuelve el valor máximo del arreglo.
78  * min($precios) → devuelve el valor mínimo del arreglo.
79  */
80 $precio_max = max($precios);
81 $precio_min = min($precios);
82
83 /*
84  * array_search() busca el primer índice cuyo valor coincida.
85  * Así obtenemos el nombre del producto más caro y el más barato
86  * usando el índice encontrado en el arreglo $productos (paralelo).
87  */
88 $idx_max = array_search($precio_max, $precios);
89 $idx_min = array_search($precio_min, $precios);
90
91 $producto_mas_caro = $productos[$idx_max];
92 $producto_mas_barato = $productos[$idx_min];
93
94 // — 8. Guardar resultados en sesión para usarlos en resultados.php —————
95 $_SESSION['inventario'] = [
96     'productos'      => $productos,
97     'precios'        => $precios,
98     'total'          => $total,
99     'promedio'       => $promedio,
100     'precio_max'     => $precio_max,
101     'precio_min'     => $precio_min,
102     'producto_mas_caro' => $producto_mas_caro,
103     'producto_mas_barato' => $producto_mas_barato,
104     'cantidad'       => $cantidad,
105 ];
106
107 // — 9. Redirigir a la página de resultados —————
108 header('Location: resultados.php');
109 exit;

```

fin procesar.php

Recibe los arreglos `nombre[]` y `precio[]`, los sanea, los almacena en `$productos[]` y `$precios[]` (paralelos), calcula todo y guarda los resultados en sesión. Luego redirige a resultados.

resultados.php

```
187 <body>
188
189 <header>
190 <h1> Resultados del Inventario</h1>
191 <p>TiendaPHP · <?= $cantidad ?> productos registrados</p>
192 </header>
193
194 <div class="container">
195
196 <!-- =====
197 | SECCIÓN 1: Tabla de productos
198 |===== -->
199 <div class="card">
200 <div class="card-header">Lista de productos</div>
201 <table>
202 <thead>
203 <tr>
204 <th>#</th>
205 <th>Producto</th>
206 <th>Precio unitario</th>
207 </tr>
208 </thead>
209 <tbody>
210 <?php
211 /*
212 * Recorremos los arreglos paralelos $productos y $precios
213 * usando el índice $i para acceder a ambos en sincronía.
214 */
215 for ($i = 0; $i < $cantidad; $i++):
216     $nombre = $productos[$i];
217     $precio = $precios[$i];
218
219     // Determinar si es el más caro o el más barato
220     $esCaro = ($precio === $precio_max && $nombre === $producto_mas_caro);
221     $esBarato = ($precio === $precio_min && $nombre === $producto_mas_barato);
222 >
223 <tr>
224 <td class="idx"><?= $i + 1 ?></td>
225 <td>
226 <?= htmlspecialchars($nombre) ?>
227 <?php if ($esCaro): ?><span class="badge badge-max">Más caro</span><?php endif; ?>
228 <?php if ($esBarato): ?><span class="badge badge-min">Más barato</span><?php endif; ?>
229 </td>
230 <td class="price">$<?= number_format($precio, 2) ?></td>
231 </tr>
232 <?php endforeach; ?>
233 </tbody>
234 </table>
235 </div>
```

```

237 <!-- =====
238 | SECCIÓN 2: Estadísticas calculadas
239 | ===== -->
240 <div class="card">
241   <div class="card-header">Estadísticas calculadas</div>
242   <div class="stats-grid">
243     <!-- Total (array_sum) -->
244     <div class="stat-card">
245       <span class="stat-label">Total inventario</span>
246       <span class="stat-value color-total">${<?= number_format($total, 2) ?></span>
247       <span class="stat-sub">array_sum($precios)</span>
248     </div>
249
250     <!-- Promedio -->
251     <div class="stat-card">
252       <span class="stat-label">Precio promedio</span>
253       <span class="stat-value color-avg">${<?= number_format($promedio, 2) ?></span>
254       <span class="stat-sub">total ÷ <?= $cantidad ?> productos</span>
255     </div>
256
257     <!-- Más caro (max) -->
258     <div class="stat-card">
259       <span class="stat-label">Producto más caro</span>
260       <span class="stat-value color-max">${<?= number_format($precio_max, 2) ?></span>
261       <span class="stat-sub"><?= htmlspecialchars($producto_mas_caro) ?></span>
262     </div>
263
264     <!-- Más barato (min) -->
265     <div class="stat-card">
266       <span class="stat-label">Producto más barato</span>
267       <span class="stat-value color-min">${<?= number_format($precio_min, 2) ?></span>
268       <span class="stat-sub"><?= htmlspecialchars($producto_mas_barato) ?></span>
269     </div>
270
271   </div>
272 </div>
273 </div>
274
275 </div><!-- /.container -->
276
277 <a href="index.php" class="btn-back">← Registrar nuevo inventario</a>
278
279 <footer>Programación Orientada a Objetos · Ingeniería Informática</footer>
280
281 </body>
282 </html>

```

fin resultados.php

Lee la sesión, muestra la tabla de productos con insignias de "Más caro / Más barato" y 4 tarjetas de estadísticas.

Resultados vistos desde el navegador:

localhost/Actividad%20Integradora%20-%20Arreglos%20unidimensionales/codi...

Aplicaciones YouTube Gmail Facebook Traducir Twitch WhatsApp Web Twitter Panel - Microsoft R...

TiendaPHP

Módulo de Gestión de Inventario - Actividad Integradora POO

REGISTRO DE PRODUCTOS

Producto 1

NOMBRE: Laptop Alienware PRECIO (\$): 35000

Producto 2

NOMBRE: Mouse Logitech G502 PRECIO (\$): 900

Producto 3

NOMBRE: Teclado Mecanico Anne Pro 2 PRECIO (\$): 2000

Producto 4

NOMBRE: Monitor Samsung monitor odyssey PRECIO (\$): 4199

Producto 5

NOMBRE: Mochila Startravel Boston Negra PRECIO (\$): 700

▶ Procesar inventario

TiendaPHP - Resultados del In...

localhost/Actividad%20Integradora%20-%20Arreglos%20unidimensionales/codi...

Aplicaciones YouTube Gmail Facebook Traducir Twitch WhatsApp Web Twitter Panel - Microsoft R...

Resultados del Inventario

TiendaPHP - 5 productos registrados

LISTA DE PRODUCTOS

#	PRODUCTO	PRECIO UNITARIO
1	Laptop Alienware Más caro	\$35,000.00
2	Mouse Logitech G502	\$900.00
3	Teclado Mecanico Anne Pro 2	\$2,000.00
4	Monitor Samsung monitor odyssey	\$4,199.00
5	Mochila Startravel Boston Negra Más barato	\$700.00

ESTADÍSTICAS CALCULADAS

TOTAL INVENTARIO \$42,799.00 array_sum(\$precios)	PRECIO PROMEDIO \$8,559.80 total ÷ 5 productos	PRODUCTO MÁS CARO \$35,000.00 Laptop Alienware	PRODUCTO MÁS BARATO \$700.00 Mochila Startravel Boston Negra
--	---	---	---

← Registrar nuevo inventario

Programación Orientada a Objetos - Ingeniería Informática

Preguntas de reflexión

a) ¿Por qué son útiles los arreglos unidimensionales? Permiten agrupar valores del mismo tipo bajo un solo nombre de variable y recorrerlos con bucles, evitando crear decenas de variables sueltas (\$producto1, \$producto2...). Hacen el código más limpio, escalable y fácil de mantener.

b) ¿Qué ventajas tienen array_sum(), max() y min()? Son funciones nativas optimizadas en C internamente, lo que las hace más rápidas y menos propensas a errores que un bucle manual. Además reducen las líneas de código y hacen la intención más legible de un vistazo.

c) ¿Cómo agregar o eliminar productos dinámicamente? Para agregar: array_push(\$productos, \$nombre) y \$precios[] = \$precio. Para eliminar: array_splice(\$productos, \$indice, 1) y lo mismo en \$precios. También podrías permitir al usuario indicar cuántos productos quiere registrar y generar el formulario dinámicamente con JavaScript.

d) ¿Qué pasaría si los arreglos no tienen la misma cantidad de elementos? Los arreglos dejarían de estar sincronizados (paralelos). Al acceder a \$productos[\$i] cuando \$precios[\$i] no existe se generaría un error Undefined offset. Por eso en procesar.php ambos arreglos se llenan en el mismo bloque condicional, garantizando que siempre crezcan juntos.

Conclusiones

¿Qué sabía? Que PHP puede recibir datos de formularios con \$_POST y que existen las variables básicas.

¿Qué aprendí? A usar arreglos paralelos para relacionar datos sin estructuras complejas, y cómo las funciones array_sum(), max(), min() y array_search() simplifican operaciones que de otro modo requerirían varios bucles.

¿Qué podría hacer con lo aprendido? Construir catálogos de productos, sistemas de calificaciones, rankings, o cualquier módulo que requiera procesar colecciones de datos relacionados de forma eficiente antes de dar el salto a bases de datos.